

02 — Wizja Architektury

[← Powrót do README](#) (./README.md) | [← Executive Summary](#) (./01-executive-summary.md) | [Następna: Architektura Integracji](#) → (./03-integration-architecture.md)



Obecna Architektura (v1.4.1)

Opis

IOC Service v1.4.1 to **monolit modułarny** z Worker Queue. Aplikacja składa się z trzech głównych komponentów: Flask API (HTTP), Background Worker (scheduled jobs) i Data Layer (PostgreSQL + Redis). Komunikacja jest synchroniczna, a integracje hardcoded w osobnych modułach service/*.

Diagram C4 — Context (Poziom 1)

```
graph TB
    subgraph "Aktorzy"
        OP[👤 Security Operator<br/>Konfiguruje feedy, zarządza systemem]
        AN[👤 Threat Analyst<br/>Wyszukuje IOC, eksportuje dane]
    end

    subgraph "Systemy Konsumenckie"
        SIEM[🖥️ SIEM Systems<br/>Splunk, Sentinel, ArcSight]
        FW[🔥 Firewalls<br/>FortiGate, Palo Alto, Checkpoint]
        APIC[📡 API Clients<br/>Skrypty, automatyzacje]
    end

    subgraph "IOC Service v1.4.1"
        IOC[🛡️ IOC Service<br/>Threat Intelligence Aggregator<br/>Flask + PostgreSQL + Redis]
    end

    subgraph "Źródła Threat Intelligence"
        CS[CrowdSec<br/>Blocklists]
        MISP_S[MISP<br/>Threat Events]
        MWDB_S[MWDB<br/>Malware Repository]
        MB[MalwareBazaar<br/>Malware Hashes]
        AC[abuse.ch<br/>ThreatFox, URLhaus,<br/>Feodo, YARAify, Hunting]
    end

    OP -->|HTTPS| IOC
    AN -->|HTTPS| IOC
    SIEM -->|HTTPS/API| IOC
    FW -->|HTTPS| IOC
    APIC -->|REST API| IOC

    IOC -.->|HTTPS/API| CS
    IOC -.->|PyMISP| MISP_S
    IOC -.->|HTTPS/API| MWDB_S
    IOC -.->|HTTPS/API| MB
    IOC -.->|HTTPS/API| AC

    style IOC fill:#4CAF50,color:#fff
```

Diagram C4 — Container (Poziom 2)

```

graph TB
    subgraph "External Clients"
        BROWSER[🌐 Web Browser]
        SIEM[📊 SIEM]
        FW[🔥 Firewalls]
    end

    subgraph "Reverse Proxy"
        NGINX["Nginx<br/>TLS 1.2+ / HTTP/2<br/>Rate Limiting<br/>Security Headers<br/>:443"]
    end

    subgraph "Application Layer"
        APP["Flask App<br/>Gunicorn Workers<br/>REST API + Web UI<br/>:8080"]
        WORKER["Background Worker<br/>Schedule Loop<br/>Feed Sync Jobs"]
    end

    subgraph "Data Layer"
        PG["PostgreSQL 16<br/>9 tabel<br/>JSONB + FTS + pg_trgm"]
        REDIS["Redis 7<br/>Cache + Rate Limiter<br/>512MB / AOF"]
    end

    subgraph "Monitoring"
        PROM["Prometheus<br/>Metrics Scraping"]
        GRAF["Grafana<br/>Dashboards"]
    end

    BROWSER --> NGINX
    SIEM --> NGINX
    FW --> NGINX
    NGINX --> APP

    APP --> PG
    APP --> REDIS
    WORKER --> PG
    WORKER --> REDIS

    APP --> PROM
    PROM --> GRAF

    style APP fill:#4CAF50,color:#fff
    style WORKER fill:#FF9800,color:#fff
    style PG fill:#2196F3,color:#fff
    style REDIS fill:#F44336,color:#fff

```

Problemy obecnej architektury

Problem	Opis	Impact	Priorytet
God Object	<code>app/main.py</code> — 2,555 LOC	Trudne testowanie, merge conflicts	● Critical
Hardcoded integracje	Każde źródło w osobnym module bez wspólnego kontraktu	2 tygodnie na nową integrację	● Critical
Brak auth /admin	Panel admina publicznie dostępny	ISO 27001 violation	● Critical
Dual schema	SQL + ORM bez synchronizacji	Schema drift risk	● Medium
Brak API versioning	Endpoints bez prefixu wersji	Breaking changes risk	● Medium
Monolityczny Config	162 LOC, 100+ pól w jednej klasie	Trudne zarządzanie	● Low

Docelowa Architektura (v2.0)

Opis

Docelowa architektura zachowuje **modularny monolit** (nie mikroserwisy — to overengineering dla tego projektu), ale wprowadza:

1. **Plugin Architecture** — adapter pattern dla integracji
2. **Service Layer** — czyste separation of concerns
3. **Security Layer** — authentication, authorization, audit
4. **Configuration Layer** — typed, grouped configuration
5. **API Layer** — versioned, documented, stable

Diagram C4 — Docelowa Container Architecture

```

graph TB
  subgraph "External Clients"
    BROWSER[🌐 Web Browser]
    SIEM[📊 SIEM]
    FW[🔥 Firewalls]
    APIC[🏠 API Clients]
  end

  subgraph "Edge Layer"
    NGINX[Nginx<br/>TLS 1.3 + mTLS<br/>Rate Limiting<br/>WAF Rules]
  end

  subgraph "Application Layer"
    subgraph "Flask App"
      AUTH[Auth Middleware<br/>OAuth2 + JWT + RBAC]
      APIV1[API v1<br/>REST Endpoints]
      WEBUI_V2[Web UI v2<br/>Workflow-centric]
      ADMIN[Admin Panel<br/>Secured + Audit]
    end

    subgraph "Service Layer"
      IND_SVC[Indicator Service<br/>CRUD, Search, Export]
      FEED_SVC[Feed Service<br/>Config, Scheduling]
      EXPORT_SVC[Export Service<br/>17 Formats + Custom]
      AUDIT_SVC[Audit Service<br/>Comprehensive Logging]
    end

    subgraph "Integration Layer 🌀"
      REGISTRY[Adapter Registry<br/>Dynamic Discovery]
      PIPELINE[Ingestion Pipeline<br/>Normalize 📦 Dedup 📦 Upsert]
      ADAPTERS[Feed Adapters<br/>CrowdSec, MISP, MWDB,<br/>abuse.ch, Malware-
Bazaar,<br/>+ Plugin Slots]
    end

    WORKER_V2[Worker v2<br/>Registry-driven<br/>Cron + On-demand]
  end

  subgraph "Data Layer"
    PG[(PostgreSQL 16<br/>Alembic-managed<br/>Single Source of Truth)]
    REDIS[(Redis 7<br/>Cache + Sessions<br/>Rate Limiter)]
  end

  subgraph "Observability"
    PROM[Prometheus]
    GRAF[Grafana]
    ALERTS[Alert Manager]
    LOGS[Structured Logs<br/>JSON + ELK]
  end

  BROWSER --> NGINX
  SIEM --> NGINX
  FW --> NGINX
  APIC --> NGINX

  NGINX --> AUTH
  AUTH --> APIV1
  AUTH --> WEBUI_V2
  AUTH --> ADMIN

  APIV1 --> IND_SVC
  APIV1 --> FEED_SVC
  APIV1 --> EXPORT_SVC

```

```

ADMIN --> AUDIT_SVC

FEED_SVC --> REGISTRY
REGISTRY --> ADAPTERS
ADAPTERS --> PIPELINE
PIPELINE --> PG

IND_SVC --> PG
IND_SVC --> REDIS
EXPORT_SVC --> REDIS

WORKER_V2 --> REGISTRY
WORKER_V2 --> PIPELINE

style REGISTRY fill:#4488ff,color:#fff
style PIPELINE fill:#4488ff,color:#fff
style ADAPTERS fill:#4488ff,color:#fff
style AUTH fill:#ff4444,color:#fff

```

Kluczowe Decyzje Architektoniczne (ADR)

ADR-001: Modularny Monolit vs. Mikroserwisy

Status: Zaakceptowana

Data: 2026-04-07

Kontekst: Aplikacja ma ~10,400 LOC i zespół 3-6 osób. Rozważano migrację do mikroserwisów.

Decyzja: Pozostajemy przy **modularnym monolicie** z czystymi granicami modułów.

Uzasadnienie:

- Zespół jest za mały na overhead mikroserwisów (service discovery, distributed tracing, API gateway)
- Latencja wewnętrzna jest krytyczna (agregacja z 10 źródeł)
- Modularny monolit daje 80% korzyści mikroserwisów przy 20% kosztów
- Przyszła migracja do mikroserwisów będzie łatwiejsza dzięki adapter pattern

Konsekwencje:

-  Prostsza infrastruktura (Docker Compose)
-  Brak problemów z distributed transactions
-  Łatwiejsze debugging i profiling
-  Trzeba pilnować granic modułów (linting, architecture tests)
-  Skalowanie pionowe, nie poziome (na razie wystarczające)

ADR-002: Adapter Pattern dla Integracji

Status: Zaakceptowana

Data: 2026-04-07

Kontekst: Dodanie nowej integracji zajmuje ~2 tygodnie. Każdy connector ma własną sygnaturę, logikę retry, normalizacji.

Decyzja: Wdrożyć **FeedAdapter Protocol** z centralnym registry i wspólnym ingestion pipeline.

Uzasadnienie:

- Standaryzacja kontraktu eliminuje duplikację (5x upsert logic, 5x deactivation)

- Runtime discovery umożliwia hot-reload konfiguracji feedów
- Fake adapters dramatycznie upraszczają testowanie
- Czas dodania integracji: 2 tygodnie → 2 dni

Szczegóły: [03-integration-architecture.md](#) (./03-integration-architecture.md)

ADR-003: Alembic jako Single Source of Truth dla Schema

Status: Zaakceptowana

Data: 2026-04-07

Kontekst: Schema zdefiniowana w 2 miejscach: SQL files i ORM models. Ryzyko schema drift.

Decyzja: Alembic migrations jako jedyne źródło prawdy. ORM models generowane z migracji.

Uzasadnienie:

- Alembic daje pełną historię zmian schema
- Automated rollback (downgrade)
- CI/CD integration (auto-migrate on deploy)
- Eliminacja database/init/*.sql (legacy)

Konsekwencje:

-  Jedna ścieżka inicjalizacji DB
 -  Automated schema drift detection w CI
 -  Wymaga migracji istniejących SQL files do Alembic
 -  PostgreSQL-specific features (triggers, functions) muszą być w Alembic
-

ADR-004: OAuth2 + Session-based Auth dla Admin

Status: Zaakceptowana

Data: 2026-04-07

Kontekst: Admin panel publicznie dostępny. Potrzebna autentykacja zgodna z ISO 27001.

Decyzja: Session-based authentication z opcjonalnym OAuth2/OIDC provider.

Uzasadnienie:

- Session-based jest prostsze dla Web UI (cookie-based)
- OAuth2/OIDC pozwala na integrację z corporate IdP (Azure AD, Keycloak)
- JWT dla API authentication (bearer tokens)
- RBAC: admin, operator, viewer

Konsekwencje:

-  ISO 27001 A.9.2.1, A.9.4.1 compliance
 -  Audit trail per user
 -  Session management (Redis backend)
 -  Token rotation i revocation
-

ADR-005: Flask vs. FastAPI

Status: Zaakceptowana (Flask remains)

Data: 2026-04-07

Kontekst: Rozważano migrację z Flask do FastAPI dla lepszego async support i auto-documentation.

Decyzja: Pozostajemy przy Flask z Flask-RESTX dla auto-documentation.

Uzasadnienie:

- Migracja Flask→FastAPI to ~3-4 tygodnie pracy bez nowej funkcjonalności
- Flask 3.x ma async view support
- Flask-RESTX daje Swagger/OpenAPI generation
- Zespół zna Flask, learning curve dla FastAPI opóźniłby delivery
- Worker jest I/O-bound (HTTP calls), nie CPU-bound — async nie da dramatycznej poprawy

Konsekwencje:

-  Brak risk migracji framework
-  Szybszy time-to-market
-  Brak natywnego async (Gunicorn workers kompensują)
-  Flask-Limiter zamiast wbudowanego middleware

ADR-006: Strategia Versioning API

Status: Zaakceptowana

Data: 2026-04-07

Kontekst: Brak wersjonowania API. Zmiany mogą łamać integracje klientów SIEM/firewall.

Decyzja: URL-based versioning (`/api/v1/`) z backward compatibility.

Uzasadnienie:

- URL versioning jest najprostsze i najbardziej czytelne
- Header-based versioning (Accept: application/vnd.ioc.v1+json) — za skomplikowane
- Stare endpointy (`/indicators` , `/export`) → redirect do `/api/v1/`

Konsekwencje:

-  Breaking changes izolowane per version
-  Klienci mogą migrować w swoim tempie
-  Maintenance dwóch wersji API w okresie przejściowym



Ewolucja: Od Obecnego Stanu do Docelowego

Faza 1: Bezpieczeństwo (M1.4.2)

Obecny stan	→	Po M1.4.2
/admin: publiczny	→	/admin: auth required
POST bez CSRF	→	CSRF tokens
Brak audit trail	→	Comprehensive audit
SECRET_KEY: ok	→	SECRET_KEY: enforced

Faza 2: Modularyzacja (M1.5.0)

Obecny stan	→	Po M1.5.0
main.py: 2,555 LOC	→	main.py: <500 LOC
ops.py: 1,529 LOC	→	admin.py + sync.py + settings.py
Inline HTML	→	Jinja templates
Logika w routes	→	Service Layer

Faza 3: Database (M1.5.1)

Obecny stan	→	Po M1.5.1
SQL files + ORM	→	Alembic only
Brak FK constraints	→	Full referential integrity
Brak PG-specific tests	→	JSONB, FTS, triggers tested

Faza 4: API (M1.6.0)

Obecny stan	→	Po M1.6.0
/indicators	→	/api/v1/indicators
Brak OpenAPI	→	Swagger UI published
Config: 1 mega-class	→	Grouped dataclasses
requirements.txt	→	pyproject.toml

Faza 5: Adaptersy 🎯 (M1.6.1)

Obecny stan	→	Po M1.6.1
5 hardcoded connectors	→	FeedAdapter Protocol
Duplikacja upsert logic	→	Shared Ingestion Pipeline
Hardcoded scheduler	→	Registry-driven scheduling
Brak capabilities	→	Runtime metadata query
2 tygodnie na integrację	→	2 dni na integrację

Faza 6: UX (M1.7.0)

Obecny stan	→	Po M1.7.0
Tech-centric UI	→	Workflow-centric UI
Admin + Business mixed	→	Separated concerns
Brak workflow guidance	→	Guided user experience



Strategia Migracji: Incremental

Dlaczego NIE Big Bang?

1. **Ryzyko** — jednorazowa migracja całego systemu to 6+ miesięcy bez widocznych efektów
2. **Feedback** — incremental delivery pozwala na korektę kursu
3. **Morale** — zespół widzi postępy co 4-6 tygodni

4. **Backward compatibility** — stare integracje działają podczas migracji

Zasady migracji

1. **Feature flags** — nowe funkcje za flagami, stary kod jako fallback
2. **Strangler fig pattern** — nowe moduły owijają stary kod, stopniowo go zastępując
3. **Parallel run** — nowe adaptory działają równolegle ze starymi connectorami
4. **Canary releases** — nowe wersje wdrażane na podzbiór feedów
5. **Rollback plan** — każdy milestone ma plan rollback (max 30 min)



Architectural Principles

P1: Separation of Concerns

Każdy moduł ma jedną, jasno zdefiniowaną odpowiedzialność. Routes nie zawierają logiki biznesowej. Services nie wiedzą o HTTP.

P2: Dependency Inversion

Moduły zależą od abstrakcji (Protocol, ABC), nie od konkretnych implementacji. Core code nigdy nie importuje bezpośrednio adaptera.

P3: Open/Closed Principle

System otwarty na rozszerzenia (nowe adaptory, nowe formaty eksportu), zamknięty na modyfikacje (core pipeline nie zmienia się przy dodaniu adaptera).

P4: Fail Fast, Fail Loud

Brakujące konfiguracje, nieprawidłowe secrets, nieosiągalne zależności — system informuje od razu, nie ignoruje cicho.

P5: Defense in Depth

Bezpieczeństwo na każdej warstwie: nginx (WAF), app (auth + CSRF), service (validation), DB (parameterized queries).

P6: Observability by Default

Każda operacja generuje metryki, logi i audit trail. Brak „cichych” operacji.

Architectural Constraints

Constraint	Uzasadnienie
Python 3.11+	Ecosystem, team expertise, PyMISP dependency
PostgreSQL 16	JSONB, FTS, proven reliability, existing data
Redis 7	Cache + rate limiter, already deployed
Docker Compose	Current infra, K8s migration optional (v2.1+)
Flask 3.x	Team expertise, migration cost too high
Modularny monolit	Team size, complexity budget
ISO 27001	Regulatory requirement

[← Executive Summary](#) (./01-executive-summary.md) | [Następna: Architektura Integracji](#) → (./03-integration-architecture.md)